

Python Programming — Take-Home Exam (Variant 2)

Handed out: 2026-07-31

Due: 2026-08-14 (23:59) — you have 2 weeks.

Rules

- Work individually. You may use the official Python documentation and the course materials. Plagiarism will be graded with 5.0 (fail).
- Submit one folder (zip or repository link) containing one `.py` file per question (`q1.py`, `q2.py`, `q3.py`, `q4.py`, `q5.py`) plus a **report (PDF, 5–8 pages)**: for each question a description of your approach, the output of your demo, and any problems you ran into; for Question 5 also your real transcript (prompt + raw model reply) and, if attempted, the bonus results. Submitting as a GitHub repository earns **+5 P**.
- Every file must run without errors with Python 3.11+ (`python q1.py`).
- Standard library only — no third-party packages. Exception: Question 5 may use the `openai` package (as in the course).
- Every file needs a short demo block at the bottom showing your functions in action.

Each question is worth 20 points (100 total).

Question 1 — Weather Log (Lists and Strings) (20 P)

A weather station logs one string per day:

```
data = ("berlin:21.5, hamburg:18.0, berlin:24.0, munich:26.5, "
        "hamburg:17.5, berlin:19.0, munich:30.0")
```

Write a function `parse_log(data: str) -> list` that turns this into a list of (city, temperature) tuples with the temperature as float.

Then write:

1. `average_temp(log, city) -> float` — average temperature of one city, rounded to 1 decimal (`round(x, 1)`). Unknown city returns 0.0.
2. `hottest_city(log) -> str` — city with the highest average.
3. `measurement_counts(log) -> dict` — how many measurements each city has.

Extra whitespace around city names or numbers must not break the parser. *Hint:* `data.split(",")` gives the entries, `entry.split(":")` separates city and temperature, `.strip()` removes surrounding whitespace — `AI_agent/rag.py` uses `split` the same way.

Question 2 — Course Schedule (Dictionaries and Sets) (20 P)

```
schedule = {
    "monday": {"python", "statistics", "english"},
    "wednesday": {"python", "databases"},
    "friday": {"statistics", "python", "presentation"},
}
```

Write functions:

1. `days_with(schedule, course)` -> `set` — all days on which a course takes place.
2. `every_day(schedule)` -> `set` — courses that happen on *every* listed day (use set operations, not counting).
3. `course_count(schedule, day)` -> `int` — number of courses on a day. An unknown day returns 0.
4. `busiest_days(schedule)` -> `list` — alphabetically sorted list of the day(s) with the most courses.

Question 3 — Fitness Tracker (Functions) (20 P)

Write a function `workout_summary(name, weight_kg, *exercises, intensity=1.0, **details):`

- `exercises`: arbitrary number of (`exercise`, `minutes`) tuples,
- calories per exercise: `minutes * 5 * intensity`, scaled additionally by `weight_kg / 70`,
- the function prints a workout report with calories per exercise,
- `details`: arbitrary keyword arguments printed as session info (e.g. `location="gym"`, `coach="Maria"`),
- if `intensity <= 0`, the function prints an error line and returns 0,
- the function **returns** the total calories, rounded to the whole number.

Show three calls: a simple workout, one with higher `intensity`, one with several `details`. Also show one call with `intensity=0`.

Question 4 — Smart Home Devices (Classes and Inheritance) (20 P)

1. Class `Device` with attributes `name`, `room`, `watts`:
 - `__init__`, `__str__` (nice output like "Lamp in living room (12 W)"),
 - `__eq__` — equal if name and room match,
 - method `daily_kwh(hours_on)` returning the consumption in kWh (`watts × hours ÷ 1000`).
2. Class `SmartLamp(Device)` with extra attribute `brightness` (0–100):
 - overrides `__str__` (adds brightness),
 - method `dim(percent)` reducing brightness, never below 0.
3. Class `House` that stores devices in a list:
 - `add(device)` — ignores the device if an equal one is already there (use your `__eq__`),
 - `devices_in(room)` -> `list`,
 - `power_hungriest()` -> `Device` — device with the most watts,
 - `__len__` so `len(house)` works — implement `__len__(self)` returning the number of stored devices.
4. Demo: at least 3 devices and 2 smart lamps, one duplicate `add` that gets ignored, and every method called once.

Question 5 — Smart Building Quiz with an LLM Judge (20 P)

Grading free-text answers takes judgment — you will make **qwen3-0.6b** the grader and defend against the judge going off the rails.

Setup: LM Studio running locally with **qwen3-0.6b** loaded, server at `http://localhost:1234/v1` — exactly like `AI_agent/rag.py` from the course. This question may use the `openai` package (`OpenAI(base_url="http://localhost:1234/v1", api_key="lm-studio")`). Always use `temperature=0`. Read only `message.content` — LM Studio puts the model's thinking into a separate `reasoning_content` field; ignore it.

Build, in this order:

1. `ask_llm(prompt: str, system: str) -> str` — sends one chat completion (system + user message) and returns the reply text.
2. `QUIZ` — a list of at least 5 (`question`, `reference_answer`) pairs about smart buildings (thermostat, CO2 sensor, presence detection, heat pump, ...) — take facts from `wiki_corpus_smart_building` or the course material. For example:

```
QUIZ = [  
    ("What does a thermostat do in a smart building?",  
     "It measures room temperature and switches heating or "  
     "cooling to hold a target temperature."),  
    ("Why do smart buildings measure CO2 concentration?",  
     "High CO2 indicates poor air quality; the ventilation "  
     "system uses it to bring in fresh air."),  
    # ... at least 3 more pairs  
]
```

3. `judge_answer(question: str, reference: str, answer: str) -> dict` — the prompt shows question, reference answer and student answer, and instructs the model to reply with **only** a JSON object

```
{"score": 7, "reason": "Right idea, misses the target-value comparison."}
```

(score: integer 0–10, reason: one sentence).

Model replies are not always valid JSON. Copy this ready-made helper into your file — it returns the parsed object, or `None` if the text is not valid JSON (you do not need to understand its inside):

```
def try_parse_json(text):  
    try:  
        return json.loads(text)  
    except json.JSONDecodeError:  
        return None
```

`judge_answer` parses the reply with `try_parse_json`. Tip: small models grade badly without a scale — anchor it in the prompt (“10 = same meaning as the reference, 7 = mostly correct, 5 = partially correct, 2 = mostly wrong, 0 = wrong or off-topic”, “different wording is fine”).

4. `check_verdict(d) -> str` — returns an error message (precise reason) if: `d` is `None`, keys are not exactly `score` and `reason`, `score` is not an integer in 0–10 (a `bool` does not count!), or

`reason` is not a non-empty string; returns the empty string "" if everything is valid. — And `judge_with_retry(question, reference, answer, max_attempts=3)` -> dict: if the error message is not empty, build a **new prompt that quotes the error message** and demands corrected JSON, then try again; after `max_attempts` failures print an error line (which question failed and why) and return `None`.

5. `grade_quiz(answers: list)` -> dict — takes one answer per quiz question, judges each with `judge_with_retry`, prints per question: score, short reason; at the end: total, percentage. If `judge_with_retry` returns `None`, count the question as **ungraded**, leave it out of the total and report it at the end. Returns a dict with `scores`, `total`, `max_possible`, `ungraded`.

Demo: grade two answer sets — one strong student (correct answers in own words) and one weak student (vague or wrong answers). The two totals must differ plausibly.

The demo block additionally contains **one real transcript** (your prompt + the raw model reply) pasted as a comment.

Bonus (+5 P): fool the judge: craft one answer that is factually **wrong but stuffed with keywords** from the reference answer, and one that is **correct but fully paraphrased** (no shared keywords). Report both scores and comment (5–8 lines): where does the 0.6b judge misjudge, and why would a larger model do better?